

MCP 服务实验指导书

实验目的

本实验旨在学习和掌握如何将现有的 Python 网页爬取代码封装为符合 Model Context Protocol (MCP) 标准的服务，并与 Cline AI 助手集成，实现通过自然语言指令获取网页内容的功能。

实验环境要求

硬件环境

- 计算机：支持 Python 运行的任何计算机
- 内存：至少 4GB RAM
- 存储：至少 1GB 可用空间

软件环境

- 操作系统：Windows 10/11, macOS 10.15+, 或 Linux
- Python 3.8 或更高版本
- Cline AI 助手（或其他支持 MCP 的客户端）

实验原理

MCP 协议简介

Model Context Protocol (MCP) 是一个开放协议，允许 AI 助手与外部服务和数据源安全连接。通过 MCP，AI 助手可以调用外部工具来扩展其功能。

网页内容获取原理

使用 `requests` 库发送 HTTP 请求获取网页原始内容，然后通过 `BeautifulSoup` 库解析 HTML，提取并清理文本内容，去除脚本、样式等无关标签。

实验步骤

第一步：环境准备

1.1 安装 Python

确保系统已安装 Python 3.8 或更高版本：

bash

```
python --version
# 或
python3 --version
```

1.2 安装必要的 Python 包

bash

```
pip install requests beautifulsoup4
```

第二步：创建 MCP 服务代码

2.0 编写爬虫代码

```
import requests
from bs4 import BeautifulSoup as bs

def get_content(url):
    header={"user-agent":"Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
    AppleWebKit/537.36 (KHTML, like Gecko) Chrome/134.0.0.0 Safari/537.36"}
    try:
        # 4.发送request请求时携带headers信息 返回response对象赋值给r
        r = requests.get(url, headers=header)
        # 5.判断状态码
        # 如果状态码=200 继续运行下面的代码
        # 不等于200 运行except部分代码
        r.raise_for_status()
        # 6.设置编码方式,解决乱码问题
        r.encoding = r.apparent_encoding
        # 7.打印网页源代码
        b=bs(r.text,"html.parser")
        print(b.text)
        return b.text
    except Exception as e:
        return f"error:{e}"

if __name__ == '__main__':
    url="https://www.douban.com"
    get_content(url)
```

借助大模型将爬虫代码封装为MCP服务,

通过fastmcp封装为MCP服务,并通过cline使用

以下为大模型回复

2.1 创建项目目录

bash

```
mkdir web-content-mcp
cd web-content-mcp
```

2.2 创建主服务文件

创建 `web_content_mcp.py` 文件，内容如下：

python

```
# `web_content_mcp.py`
#!/usr/bin/env python3
"""
测试 MCP 服务器
"""

import requests
from bs4 import BeautifulSoup as bs
from mcp.server.fastmcp import FastMCP
# 创建MCP服务器实例
mcp = FastMCP("Web Scraper")

@mcp.tool()
def get_web_content(url: str) -> str:
    """获取网页内容"""
    header = {
        "user-agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/134.0.0.0 Safari/537.36"
    }
    try:
        r = requests.get(url, headers=header, timeout=10)
        r.raise_for_status()
        r.encoding = r.apparent_encoding
        soup = bs(r.text, "html.parser")
        text_content = soup.get_text(separator='\n', strip=True)
        return text_content[:1000] # 限制返回长度
    except Exception as e:
        return f"Error: {e}"

if __name__ == "__main__":
    print("启动 MCP 服务器...")
    mcp.run(transport='stdio')
```

第三步：测试 MCP 服务

3.1 基础功能测试

bash

```
mcp dev web_content_mcp.py
```

运行后，如图所示。

```
(.venv) (base) zhaoqing@bogon MCP_Project % mcp dev test.py
Starting MCP inspector...
Proxy server listening on 127.0.0.1:6277
Session token: 8c10f0446e9ffd7b36066770fa8e50f5b045c634aae7d2989b91b9a395d84a03
Use this token to authenticate requests or set DANGEROUSLY_OMIT_AUTH=true to disable auth

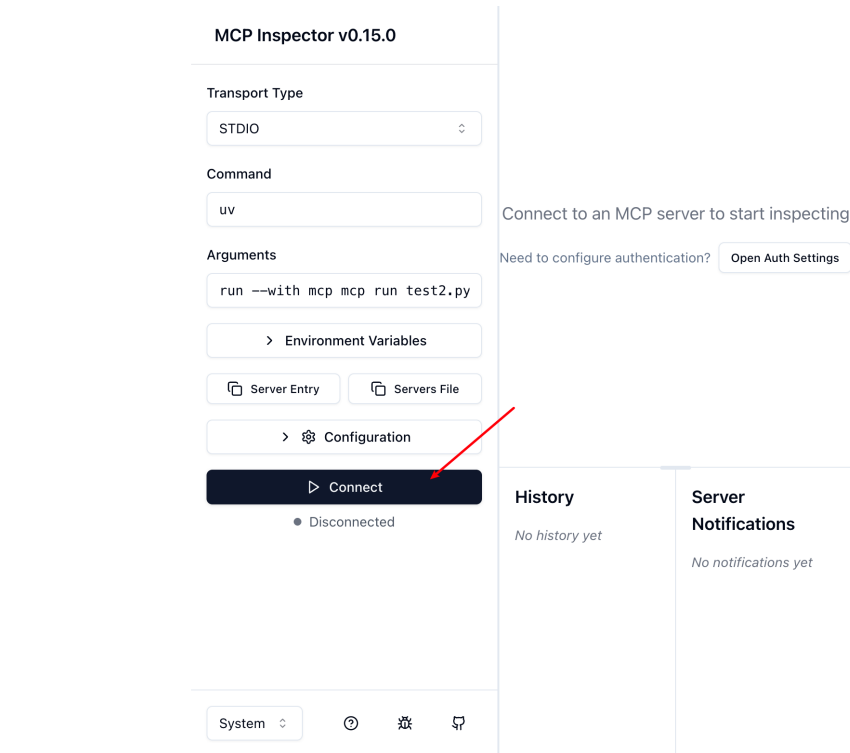
Open inspector with token pre-filled:
http://localhost:6274/?MCP\_PROXY\_AUTH\_TOKEN=8c10f0446e9ffd7b36066770fa8e50f5b045c634aae7d2989b91b9a395d84a03

MCP Inspector is up and running at http://127.0.0.1:6274
New STDIO connection request
```

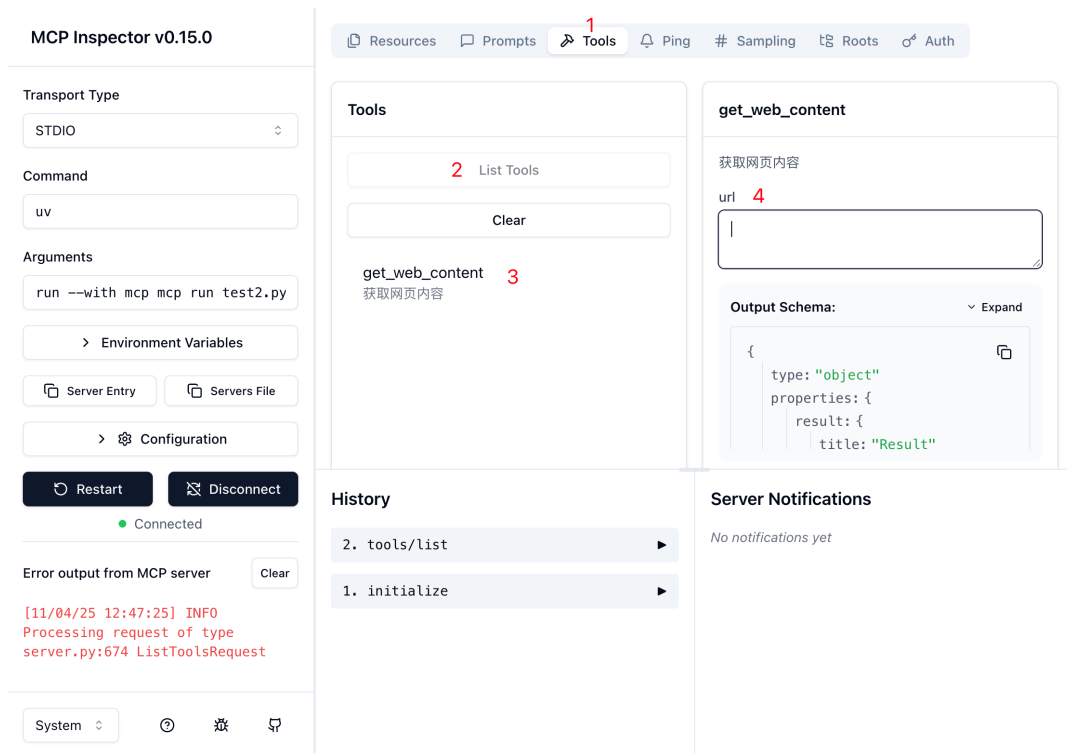
浏览器打开链接，点击Connect（注意电脑中需要有uv指令）

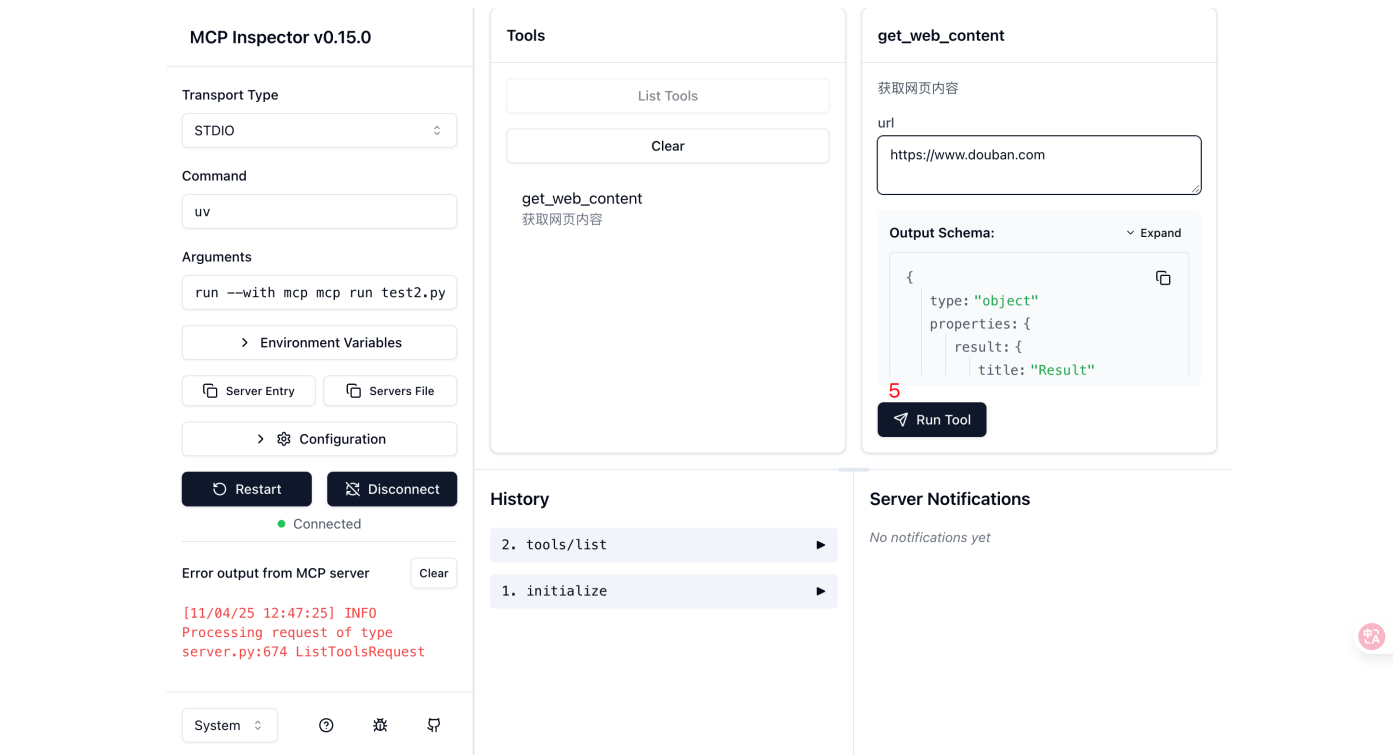
```
zhaoqing — -zsh — 80x24

Last login: Tue Oct 28 13:06:08 on console
[(base) zhaoqing@bogon ~ % brew install uv
==> Auto-updating Homebrew...
Adjust how often this is run with HOMEBREW_AUTO_UPDATE_SECS or disable with
HOMEBREW_NO_AUTO_UPDATE. Hide these hints with HOMEBREW_NO_ENV_HINTS (see `man b
rew`).
==> Downloading https://mirrors.tuna.tsinghua.edu.cn/homebrew-bottles//bottles-p
ortable-ruby/portable-ruby-3.4.5.arm64_big_sur.bottle.tar.gz
##### 100.0%
==> Pouring portable-ruby-3.4.5.arm64_big_sur.bottle.tar.gz
==> Auto-updated Homebrew!
Updated 3 taps (homebrew/services, homebrew/core and homebrew/cask).
==> New Formulae
ab-av1: AV1 re-encoding using ffmpeg, svt-av1 & vmaf
abpoa: SIMD-based C library for fast partial order alignment using adaptive band
acme.sh: ACME client
acronym: Python-based tool for creating English-ish acronyms from your fancy pro
ject
act_runner: Action runner for Gitea based on Gitea's fork of act
ad: Adaptable text editor inspired by vi, kakoune, and acme
adapterremoval: Rapid adapter trimming, identification, and read merging
adaptivecpp: SYCL and C++ standard parallelism for CPUs and GPUs
add-determinism: Build postprocessor to reset metadata fields for build reproduc
ibility
```

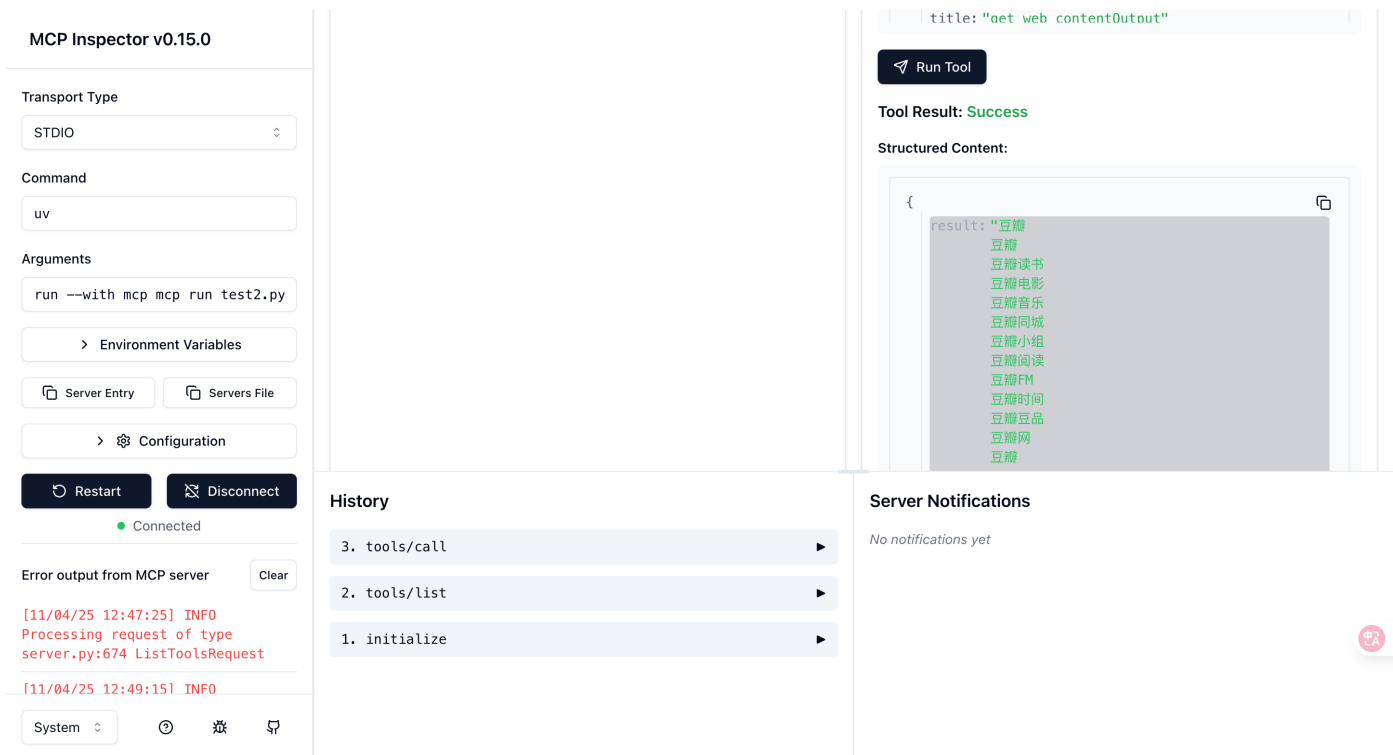


选择服务，输入url，点击运行





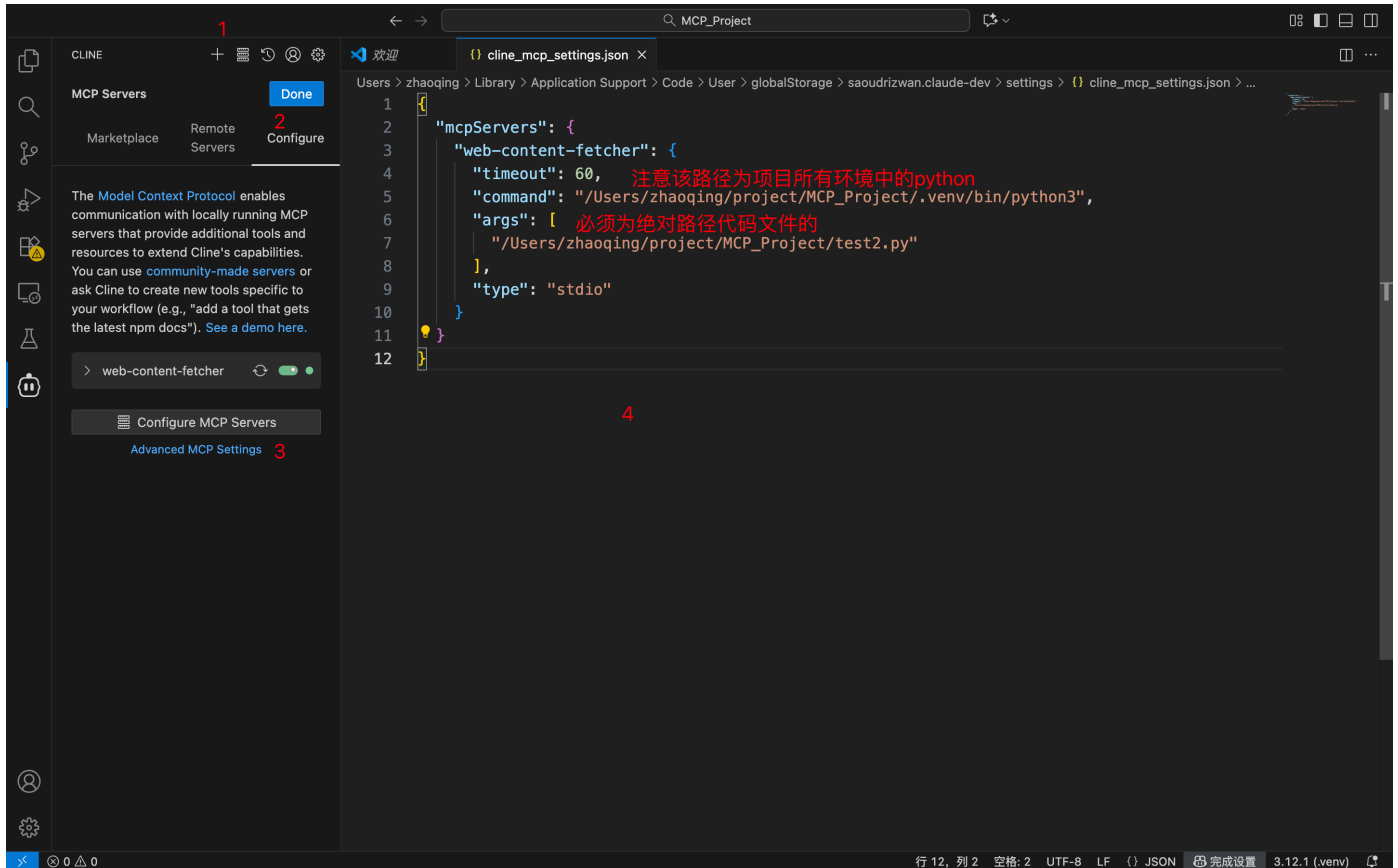
运行结果如下：



第四步：配置 Cline 使用 MCP 服务

4.1 找到 Cline 配置目录

4.2 创建或编辑 MCP 配置文件



json

```
{
  "mcpServers": {
    "web-content-fetcher": {
      "command": "python",
      "args": ["/绝对路径/到/你的/web_content_mcp.py"]
    }
  }
}
```

注意：需要将 /绝对路径/到/你的/ 替换为实际的绝对路径，保存后会出现绿色圆点，代表配置成功

第五步：在 Cline 中使用网页内容获取功能

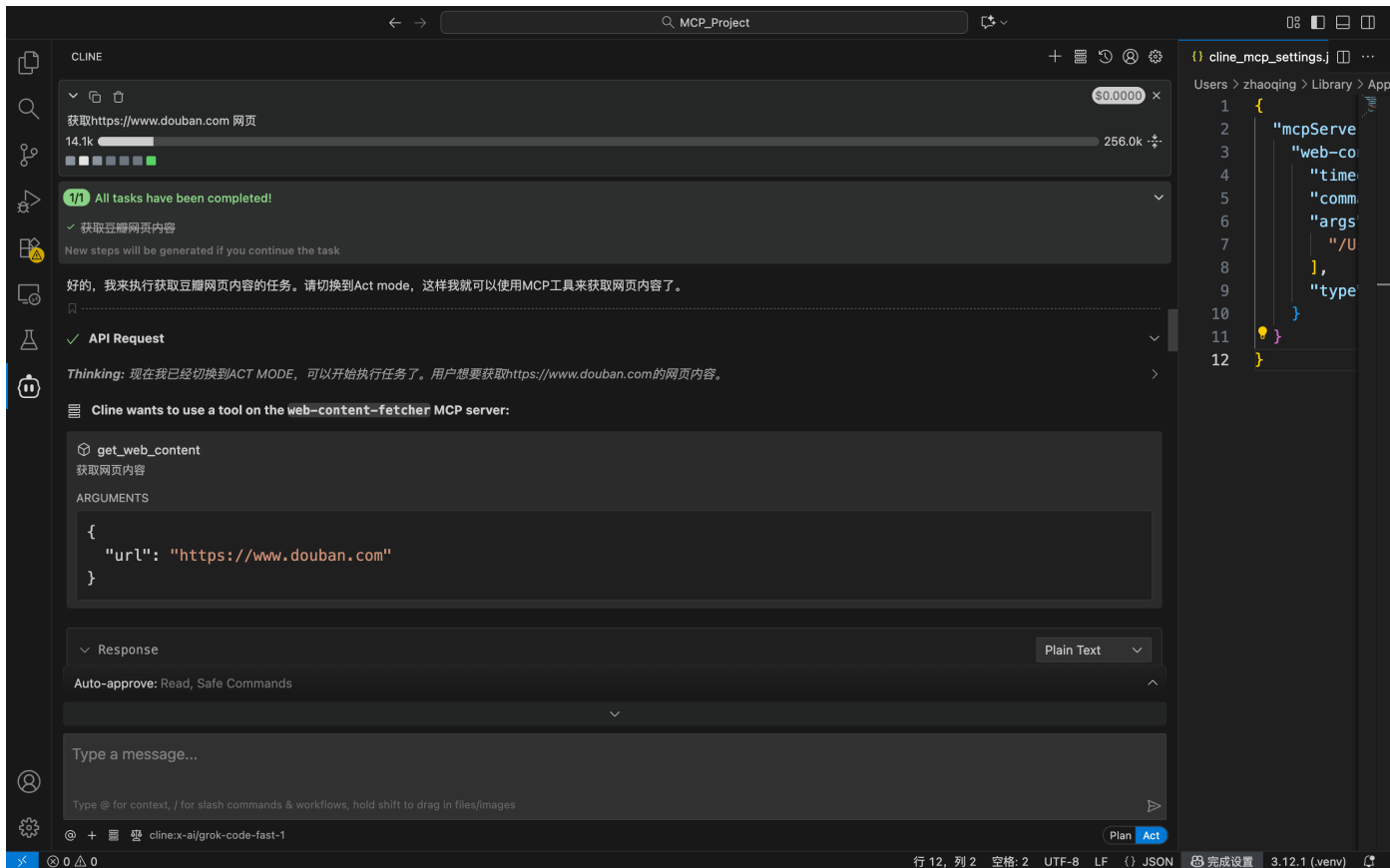
5.1 基本使用

在 Cline 聊天界面中，你可以直接请求获取网页内容：

text

```
用户：请帮我获取 https://example.com 的网页内容
```

Cline 会自动识别并使用 `get_web_content` 工具，返回格式化的网页内容。



第六步：封装为docker镜像

6.1 编写Dockerfile

dockerfile

```
# 使用官方 Python 镜像
FROM python:3.11-slim

# 设置工作目录
WORKDIR /app

# 安装系统依赖
RUN apt-get update && apt-get install -y \
    curl \
    && rm -rf /var/lib/apt/lists/*

# 复制 requirements 文件
COPY requirements.txt .

# 安装 Python 依赖
RUN pip install --no-cache-dir -r requirements.txt

# 复制应用代码
COPY test_server.py .

# 创建非root用户
RUN useradd -m -u 1000 mcpuser && chown -R mcpuser:mcpuser /app
```



```
USER mcpuser

# 设置环境变量
ENV PYTHONUNBUFFERED=1
ENV PYTHONPATH=/app

# 启动 MCP 服务器
CMD ["python", "test_server.py"]
```

6.2 导出项目环境requirements.txt

```
requests>=2.31.0
beautifulsoup4>=4.12.0
mcp>=1.0.0
```

6.3 构建镜像

```
docker build -t mcp-web-scraper .
```

6.4 运行容器

```
docker run -it --rm mcp-web-scraper
```

第七步 将镜像上传至阿里云